SAI VIDYA INSTITUTE OF TECHNOLOGY

Rajanukunte, Via Yelahanka, Bengaluru – 560064

INDUSTRIAL INTERNSHIP REPORT

on

QUICKLINK – URL SHORTENER

A Web-Based Application for Shortening URLs and Tracking Clicks

Student Name	Sandeep S
Roll Number	1VA25CS127
Department	Computer Science and Engineering (CSE)
Internship Duration	6 Weeks
Internship Start Date	06 August 2026
Internship Period	06 August 2026 – 16 September 2026
Development Platform	Lovable
Live Application	url-swift-paste.lovable.app
GitHub Repository	github.com/sandeeps05092006-creator/upskillcampus

Submitted as part of the 6-week industrial internship and project program.

Executive Summary

This report presents the six-week industrial internship work carried out on **QuickLink – URL Shortener**, a web-based application developed to convert long web addresses into concise, shareable links while maintaining basic click tracking and link history. The project was developed using Lovable, with the source-code workflow connected to GitHub.

The application provides four principal views—Home, Dashboard, History and About. A submitted HTTP/HTTPS URL is validated, a short code is generated, collision checking is performed, and the URL mapping is stored. Opening a short link redirects the visitor to the original destination while updating the click count. The Dashboard summarizes usage, while History provides search, copy, open and delete actions.

The supplied project evidence records **9 URLs shortened, 8 total clicks and an average of 0.9 clicks per link** on the Dashboard. The internship provided practical exposure to software requirements, user-interface design, validation, routing, persistent data handling, functional testing, documentation and version control.

Report objective

The objective is to document the problem addressed, proposed solution, system design, interfaces, testing approach, observed outcome, learning gained and future enhancement scope, following the organization of the supplied USC/UCT internship-report sample.

Student: Sandeep S | Roll No.: 1VA25CS127 | Department: CSE | Institution: Sai Vidya Institute of Technology

Table of Contents

The page sequence is intentionally organized to closely follow the supplied sample report structure.

1 Preface

This report summarizes the complete six-week work undertaken for QuickLink – URL Shortener. The project was approached as a practical software-engineering exercise in which a real usability problem—sharing long and unwieldy web addresses—was translated into a working web application.

The work involved understanding the problem, planning the user flow, implementing URL validation, generating unique short codes, maintaining stored URL records, handling redirection, tracking clicks, presenting dashboard statistics, providing history management and documenting a small JSON API.

The internship was valuable because it connected academic concepts with an end-to-end software product. Rather than treating the interface, application logic, data handling and documentation as separate tasks, the project required them to work together as one system.

Acknowledgement

I sincerely thank the internship coordinators, mentors and faculty members who supported the learning process and provided guidance throughout the project. I also acknowledge Sai Vidya Institute of Technology for providing an academic environment that encourages practical learning and technical skill development.

Message to juniors and peers

A project becomes more valuable when implementation, testing, documentation and source-code organization are treated as one complete engineering activity. Building a small but complete application is a useful way to understand how individual software concepts connect in practice.

2 Introduction

2.1 About Sai Vidya Institute of Technology

Sai Vidya Institute of Technology (SVIT) is located at Rajanukunte, Via Yelahanka, Bengaluru, Karnataka. The institute's official website states that SVIT is affiliated to Visvesvaraya Technological University (VTU), approved by AICTE and recognized by the Government of Karnataka. The institute was established in 2008 by Sri Sai Vidya Vikas Shikshana Samithi.

SVIT emphasizes academic excellence, innovation, research, industry-oriented learning and holistic development. Its academic portfolio includes Computer Science and Engineering and other engineering and management programs.

2.2 About the QuickLink Project

QuickLink is a web-based URL-shortening application. Its primary purpose is to turn long web addresses into shorter links that are easier to share. The project includes Home, Dashboard, History and About views, together with a small JSON developer API.

The supplied project documentation describes validation of HTTP/HTTPS input, 6–8 character short-code generation using cryptographic randomness, collision checking, persistence of the URL mapping, click counting and redirect handling.

2.3 Objectives

• Create a simple web interface for shortening long URLs.

• Generate short and unique codes for stored URLs.

• Validate submitted HTTP and HTTPS URLs.

• Store the original URL, short code, click count and creation time.

• Redirect visitors from a short link to the original destination while incrementing the click count.

• Provide Dashboard and History views for monitoring created links.

• Provide a developer API for programmatic URL shortening and retrieval.

• Connect the project source-code workflow with GitHub for version control and submission.

2.4 References

1. Sai Vidya Institute of Technology – official website: https://saividya.ac.in/

2. QuickLink live application: https://url-swift-paste.lovable.app

3. QuickLink project documentation and supplied interface screenshots.

4. Sample Industrial Internship Report structure supplied for the internship.

2.5 Glossary

Term	Meaning
URL	Uniform Resource Locator
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
UI	User Interface
GitHub	Web-based platform for source-code hosting and version control
Redirect	Forwarding a visitor from a short URL to its original destination
Short code	Compact identifier used to locate a stored original URL

3 Problem Statement

Long web addresses are often inconvenient to share in messages, presentations, emails and printed material. They may be difficult to read, copy accurately and remember. The project addresses this usability problem by providing a web application that accepts a long URL and produces a shorter link that can redirect the visitor to the original destination.

The engineering problem is not limited to shortening the visible text. The application must also validate input, generate a unique identifier, persist the relationship between the short code and original URL, count visits, provide a usable management interface and expose enough information for monitoring and programmatic use.

The supplied evidence shows that QuickLink addresses these requirements through URL validation, unique short-code generation with collision checking, persistent URL mapping, redirect-and-count behavior, Dashboard statistics, History management and a documented JSON API.

Key requirements

• Input should accept well-formed HTTP/HTTPS URLs.

• Generated short codes should be unique among stored links.

• The mapping between short code and original URL should persist.

• Opening a short link should redirect to the original destination and update the click counter.

• Users should be able to inspect and manage created links.

• The system should provide useful aggregate statistics.

• The API should provide a programmatic path for shortening and retrieving URL data.

4 Existing and Proposed Solution

4.1 Existing Solution / Common Approach

A basic way of sharing a web resource is to send the complete original URL. This requires both sender and receiver to handle the full address. Another common approach is to use a URL-shortening service; however, implementations vary in the visibility they provide into stored links, click counts, history and developer access.

4.2 Proposed Solution – QuickLink

QuickLink proposes a focused solution consisting of a simple shortening interface, validated HTTP/HTTPS input, unique short-code generation, persistent mapping, click tracking, Dashboard statistics, History management and a small JSON API.

4.3 Value Addition

• Short links are easier to share and visually cleaner.

• Click tracking provides basic usage statistics.

• Dashboard and History views centralize created links.

• URL validation improves input quality.

• Collision checking reduces the risk of duplicate short codes.

• Developer API support allows programmatic integration.

4.4 Code Submission (GitHub Link)

GitHub repository: https://github.com/sandeeps05092006-creator/upskillcampus

Because QuickLink is a web application rather than the Java example used in the sample instructions, the repository link is used as the source-code submission location. The actual Lovable-connected project source is stored in this repository.

4.5 Report Submission (GitHub Link)

Final report filename: **QuickLink_Sandeep_S_USC_UCT.pdf**

Planned GitHub report URL:
https://github.com/sandeeps05092006-creator/upskillcampus/blob/main/QuickLink_Sandeep_S_USC_UCT.pdf

Upload this PDF to the repository root using the filename above. After upload, the report URL becomes the working submission link.

5 Proposed Design / Model

The system is organized as a request-to-storage-to-redirect pipeline. A user submits a URL through the Home interface. The application validates the URL, creates a unique short code, checks for collisions, stores the mapping and returns the short URL. When the short URL is opened, the application finds the stored destination, increments the click counter and redirects the visitor.

5.1 High Level Diagram

USER	HOME / INPUT VALIDATION	SHORT-CODE + STORAGE	REDIRECT / TRACKING	DASHBOARD / HISTORY
Submit URL	HTTP/HTTPS check	Generate + persist	Increment clicks + redirect	View / search / copy / open / delete

Figure 1: High-level flow of the QuickLink system.

5.2 Low Level Diagram

CLIENT	API / SERVER ROUTE	DATA LAYER	REDIRECT ROUTE
URL input + validation	Shorten request / list	URL, short code, click count, creation time	Lookup code → increment click count → redirect

Figure 2: Low-level logical architecture.

5.3 Interfaces and Developer API

Home Interface

The Home page provides the main URL input field and the Shorten URL action. The interface communicates the core purpose of QuickLink immediately and provides short-link generation as the primary task.

Dashboard Interface

The Dashboard provides total URLs shortened, total clicks, average clicks per link and a list of recently created URLs. The supplied evidence shows 9 total URLs shortened, 8 total clicks and an average of 0.9 clicks per link.

History Interface

The History page provides a searchable table of stored links and actions for Copy, Open and Delete. It displays original URLs, short codes, click counts and creation timestamps.

About Interface

The About page explains the application workflow and provides developer API documentation. The documented endpoints include **POST /api/public/short** for shortening and **GET /api/public/urls** for retrieving totals and stored links.

Developer API example

```
POST /api/public/short
Content-Type: application/json

{ "url": "https://example.com/a/very/long/link" }

Response includes original_url, short_code and short_url.
```

GET /api/public/urls returns totals plus stored links for Dashboard and History views.

6 Performance Test

Performance testing for this project focuses on functional constraints that affect reliability and usability: input validation, uniqueness of short codes, correct redirect behavior, click-count updates, Dashboard statistics, History search and deletion.

6.1 Test Plan / Test Cases

Test Case	Action	Expected Result	Evidence / Status
Valid HTTP/HTTPS URL	Submit a well-formed URL	Short URL generated	Supported by documented workflow
Invalid URL	Submit malformed/non-HTTP(S) input	Validation error shown	Supported by About-page description
Unique short code	Create multiple links	Codes should not collide	Collision check documented
Short-link redirect	Open generated short link	Redirect to original + increment click count	Workflow documented
Dashboard statistics	Create/open links	Totals and average update	Dashboard populated in supplied evidence
History search	Search by URL or short code	Matching records displayed	Search field visible
History delete	Use Delete on a stored link	Selected record removed	Delete control visible

6.2 Test Procedure

• Open the deployed QuickLink application.

• Enter a valid HTTP/HTTPS URL and submit it.

• Record the generated short code and open the resulting short link.

• Check whether the destination is reached and the click count changes.

• Open Dashboard and verify the aggregate statistics.

• Open History and verify search, copy, open and delete controls.

• Repeat with invalid URL input to confirm validation behavior.

6.3 Performance Outcome

The supplied Dashboard evidence shows **9 total URLs shortened**, **8 total clicks** and an **average of 0.9 clicks per link**. The recent-links table also shows stored original URLs, short codes, click counts, creation timestamps and Copy/Open controls. These values demonstrate that the application is maintaining link records and presenting aggregate usage information.

The performance discussion is intentionally based on the supplied functional evidence. No claims are made here about load testing, response-time benchmarks or large-scale production capacity because those measurements were not supplied.

7 My Learnings

• Understanding the complete workflow of a URL-shortening application.

• Understanding client-side and server-side URL validation.

• Understanding short-code generation and collision checking.

• Working with persistent URL records and click counters.

• Designing Dashboard and History interfaces around stored data.

• Understanding a JSON-based developer API.

• Understanding deployment through Lovable and connecting the source-code workflow with GitHub.

• Recognizing the importance of functional testing, evidence collection and technical documentation.

The project also improved problem-solving by requiring the interface, application logic, data persistence and documentation to work together as a single system. This experience is useful for future software-development work because it emphasizes not only implementation, but also testing, version control and communication of technical decisions.

Career relevance

The experience provides a foundation for future work in web development, software engineering, API integration, database-backed applications, deployment workflows and technical documentation. It reinforces the value of converting a requirement into a clearly testable workflow.

8 Future Work Scope

The following are proposed enhancements for future versions and are not claimed as current features.

• User accounts and authenticated link management.

• Custom aliases for human-readable short links.

• QR-code generation for shortened URLs.

• Richer analytics and reporting.

• Link expiration and access controls.

• Additional monitoring and operational metrics.

• More advanced API authentication and rate limiting.

• Improved accessibility and responsive behavior across a wider range of devices.

9 Conclusion

QuickLink demonstrates a practical web application for converting long URLs into short links while maintaining click tracking and link history. The project combines a simple user interface with URL validation, unique short-code generation, persistent mapping, redirect handling, Dashboard statistics, History management and a developer API.

The supplied application evidence shows a clear Home page, a statistics-oriented Dashboard, a searchable History page and an About page explaining the system workflow and API. The project therefore provides a compact example of how a real-world software requirement can be translated into a working web application and documented as an engineering project.

The project can be further extended with user accounts, custom aliases, QR codes, richer analytics, expiration, access controls and additional monitoring. These enhancements would make the system more suitable for broader production use while preserving the core shortening-and-redirect workflow.

10 Project Evidence / User Interface

The following screenshots are included as project evidence from the supplied QuickLink application.

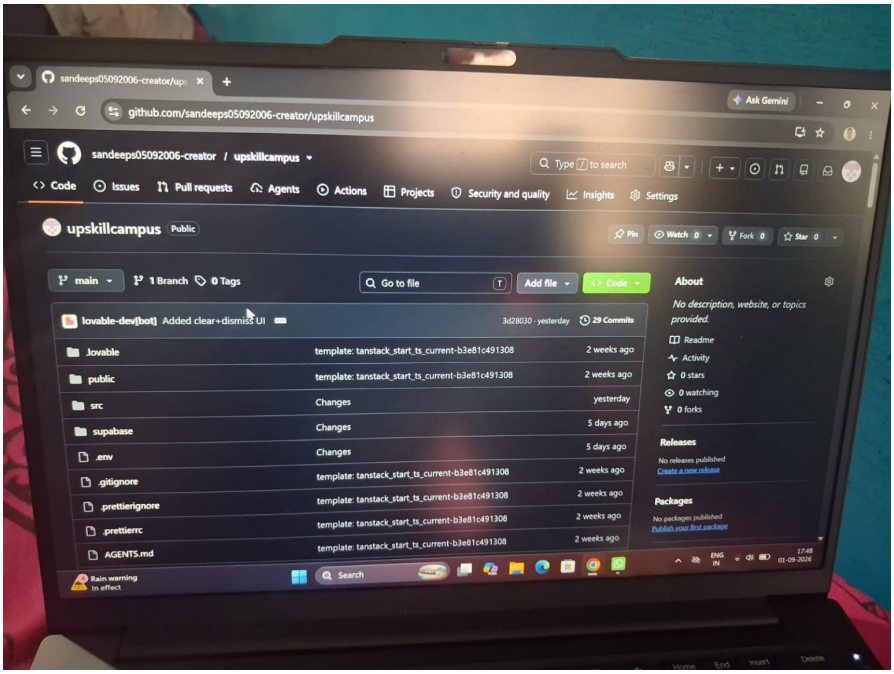

Figure 3: QuickLink Home interface — URL entry and shortening workflow.

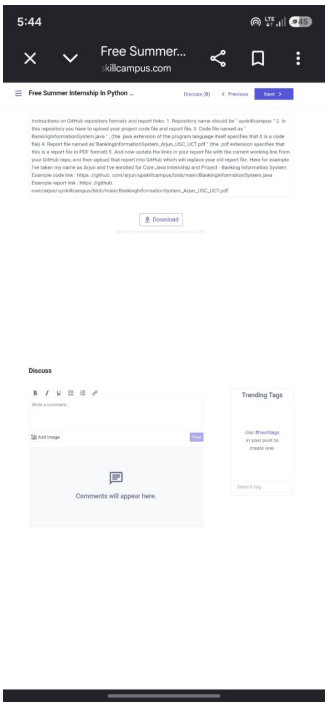

Figure 4: QuickLink Dashboard — totals, clicks, average and recent links.

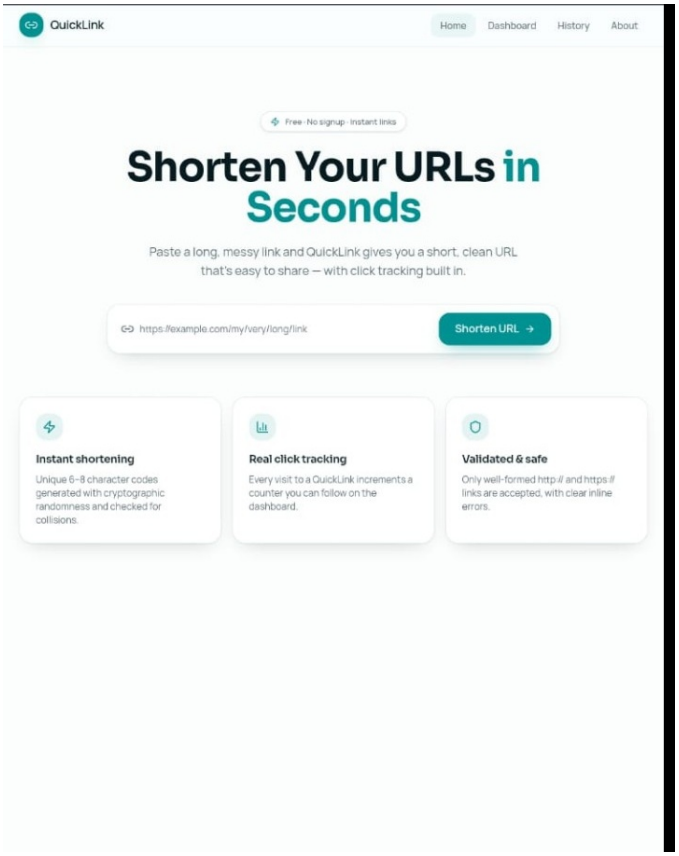

Figure 5: QuickLink History — search, copy, open and delete controls.

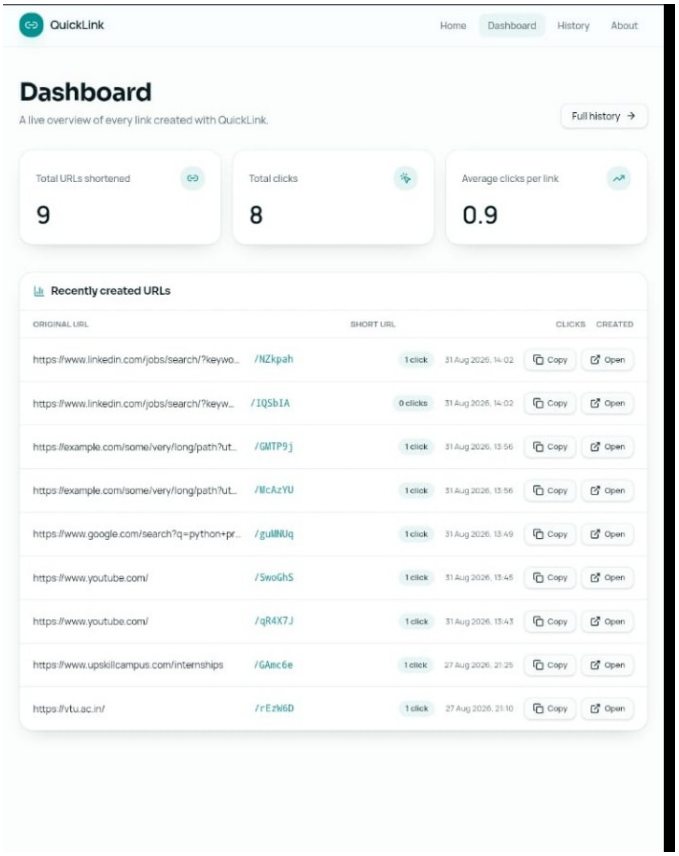

Figure 6: QuickLink About — workflow and developer API documentation.

11 Project Links and Submission Details

Item	Final value
Student Name	Sandeep S
Roll Number	1VA25CS127
Department	Computer Science and Engineering (CSE)
Institution	Sai Vidya Institute of Technology
Internship Duration	6 Weeks
Start Date	06 August 2026
End Date	16 September 2026
Live Website	https://url-swift-paste.lovable.app
GitHub Repository	https://github.com/sandeeps05092006-creator/upskillcampus
Final Report Filename	QuickLink_Sandeep_S_USC_UCT.pdf
Report GitHub URL	https://github.com/sandeeps05092006-creator/upskillcampus/blob/main/QuickLink_Sandeep_S_USC_UCT.pdf

Final GitHub submission checklist

• Keep the repository name exactly as upskillcampus.

• Keep the repository public so the evaluator can open it.

• Ensure the actual Lovable-connected QuickLink source is present in the repository.

• Upload this PDF at the repository root as QuickLink_Sandeep_S_USC_UCT.pdf.

• After uploading the PDF, open the report URL once to confirm it works.

• Do not use the sample's BankingInformationSystem.java filename for this web project; submit the actual QuickLink source repository instead.

Submission-ready status: REPORT CONTENT AND LINKS HAVE BEEN PREPARED. The remaining action is to upload this exact PDF to the root of your existing public upskillcampus repository.

Live application: https://url-swift-paste.lovable.app

GitHub: https://github.com/sandeeps05092006-creator/upskillcampus